

Roll no :- S002	Name :- Kaif Ansari
Batch :- 1	Class :- SY.BSC.IT
Date of assignment :- 25/08/2026	Date of submission:-25/08/2026

PRACTICAL 21

Aim:- Join two CSV files in Spark DataFrames based on a common column and write the output to a file.

Input:-

```
ThisBuild / scalaVersion := "2.12.18"

libraryDependencies += Seq(
  "org.apache.spark" %% "spark-core" % "3.5.1",
  "org.apache.spark" %% "spark-sql" % "3.5.1"
)
lazy val root = (project in file("."))
  .settings(
    name := "kaif"
  )
```

```
build.sbt  Kaif.scala  file1.csv  file2.csv
1  import org.apache.spark.sql.SparkSession
2
3  object Kaif extends App {
4    val s = SparkSession.builder()
5      .appName("JoinTwoCSV")
6      .master("local[*]")
7      .getOrCreate()
8
9    val d1 = s.read
10     .option("header", "true")
11     .csv("file1.csv")
12
13    val d2 = s.read
14     .option("header", "true")
15     .csv("file2.csv")
16
17    val j = d1.join(d2, Seq("id"), "inner")
18
19    j.show()
20
21    j.write
22     .option("header", "true")
23     .csv("output_result")
24
25    s.stop()
26  }
```

```
build.sbt Kaif.scala file1.csv file2.csv
1 id,name
2 1,Kaif
3 2,Aditya
4 3,Aryan
```

```
build.sbt Kaif.scala file1.csv file2.csv
1 id,score
2 3,80
3 2,70
4 1,60
```

Output:-

```
+---+-----+-----+
| id|  name|score|
+---+-----+-----+
| 1|  kaif|  60|
| 2|Aditya|  70|
| 3| Aryan|  80|
+---+-----+-----+
```

PRACTICAL 22

Aim:- Set up Apache Spark locally and count the frequency of words in a text file.

Input:-

```
build.sbt x D22.scala input.txt
1 ThisBuild / scalaVersion := "2.12.18"
2
3 libraryDependencies += Seq(
4   "org.apache.spark" %% "spark-core" % "3.5.1",
5   "org.apache.spark" %% "spark-sql" % "3.5.1"
6 )
7
8 lazy val root = (project in file("."))
9   .settings(
10     name := "D22"
11   )
12
```

```
build.sbt x D22.scala x input.txt
1 import org.apache.spark.sql.SparkSession
2
3 object D22 extends App
4 {
5     val s = SparkSession.builder()
6         .appName("WordCountExample")
7         .master("local[*]")
8         .getOrCreate()
9
10    val r = s.sparkContext.textFile("input.txt")
11    val w = r.flatMap(line => line.split("\\s+"))
12    val wo = w.map(word => (word, 1))
13    val wc = wo.reduceByKey(_ + _)
14    wc.collect().foreach(println)
15    s.stop()
16 }
17
18
```

Output:-

```
26/08/25 11:30:01 INFO SparkUI: Stopped Spark web UI at http://DESKTOP-6F06NCQ:4040
(is,2)
(easy,1)
(powerful,1)
(Spark,2)
26/08/25 11:30:01 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint
on /dev/loop6 at 78.81.5158.44:8080. Host: 192.168.1.101. Port: 8080.
```

PRACTICAL 23

Aim:- Filter rows in a CSV file using spark DataFrames where a numeric column exceeds a threshold.

Input:-

```
build.sbt x d27.scala x superstore.csv
1 ThisBuild / scalaVersion := "2.12.18"
2
3 libraryDependencies += Seq(
4     "org.apache.spark" %% "spark-core" % "3.5.1",
5     "org.apache.spark" %% "spark-sql" % "3.5.1"
6 )
7 lazy val root = (project in file("."))
8     .settings(
9         name := "d24"
10    )
11
```

```

build.sbt  d27.scala  superstore.csv
1      import org.apache.spark.sql.SparkSession
2
3  ▶    object d27 extends App {
4        val spark = SparkSession.builder()
5          .appName("FilterCSVExample")
6          .master("local[*]")
7          .getOrCreate()
8
9        import spark.implicits._
10
11       val df = spark.read
12         .option("header", "true")
13         .option("inferSchema", "true")
14         .csv("src/main/resources/superstore.csv")
15
16       val f = df.filter($"Sales" > 10000)
17
18       println("Filtered Data:")
19       f.show()
20       spark.stop()
21     }
22

```

Output:-

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code
2624	CA-2017-127180	10/22/2017	10/24/2017	First Class	TA-21385	Tom Ashbrook	Home Office	United States	New York City	New York	10024
2698	CA-2014-145317	3/18/2014	3/23/2014	Standard Class	SM-20320	Sean Miller	Home Office	United States	Jacksonville	Florida	32216
4191	CA-2017-166709	11/17/2017	11/22/2017	Standard Class	HL-15040	Hunter Lopez	Consumer	United States	Newark	Delaware	19711
6827	CA-2016-118689	10/2/2016	10/9/2016	Standard Class	TC-20980	Tamara Chand	Corporate	United States	Lafayette	Indiana	47905
8154	CA-2017-140151	3/23/2017	3/25/2017	First Class	RB-19360	Raymond Buch	Consumer	United States	Seattle	Washington	98115

Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	11199.968	4	0.2	3919.9888
South	TEC-MA-10002412	Technology	Machines	Cisco TelePresenc...	22638.48	6	0.5	-1811.0784
East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	10499.97	3	0	5039.9856
Central	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	17499.95	5	0	8399.976
West	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	13999.96	4	0	6719.9808

PRACTICAL 24

Aim:- Perform a group-by operation in spark DataFrames to compute the average of each group.

Input:-

```
build.sbt  d27.scala x  superstore.csv
1  import org.apache.spark.sql.SparkSession
2  import org.apache.spark.sql.functions._
3
4  object Demo extends App {
5      val spark = SparkSession.builder()
6          .appName("AverageSales")
7          .master("local[*]")
8          .getOrCreate()
9
10     val df = spark.read
11         .option("header", "true")
12         .option("inferSchema", "true")
13         .csv("src/main/resources/superstore.csv")
14
15     val result = df.groupBy("Category")
16         .agg(avg("Sales").alias("Average Sales"))
17
18     println("Average Sales by Category:")
19     result.show()
20
21     spark.stop()
22 }
23
```

Output:-

```
+-----+-----+
|   Category| Average Sales|
+-----+-----+
|Office Supplies|121.69225531914947|
|   Furniture| 353.4459311957568|
|   Technology| 454.5405475802047|
+-----+-----+
```